Final Revision: Selected Problems and Solutions

ELG 5218 – Uncertainty Evaluation in Engineering Measurements and Machine Learning

Instructor: Miodrag Bolić, University of Ottawa

April 2026

Abstract

This document provides a set of revision problems for the final class of ELG 5218 (Winter 2026). **Part I** selects one representative problem from each of the 11 course topics. **Part II** introduces *cross-topic bridging problems*.

Contents

Part I
Representative Review Problems (One Per Topic)

1 Topic 1: Probabilistic Reasoning & Bayesian Foundations

R1: Conjugacy and the Shrinkage of Epistemic Uncertainty

A coin has unknown bias $\theta \sim \text{Beta}(2, 2)$. You observe $n = 10$ flips with $k = 7$ heads.
(a) Write the posterior. Compute the posterior mean, prior mean, and MLE.
(b) Show the posterior mean lies between the prior mean and MLE.
(c) What happens as $n \to \infty$? Relate to epistemic uncertainty.

Solution

(a) Revision and derivation. The Beta–Binomial model is the simplest example of Bayesian conjugate updating. Conjugacy means that if the prior belongs to a particular family, the posterior belongs to *the same family* with updated parameters.
The likelihood for k heads in n independent flips is

$$p(k \mid \theta) = \binom{n}{k} \theta^k (1-\theta)^{n-k}.$$

When we multiply this by the prior $p(\theta) = \text{Beta}(\alpha_0, \beta_0) \propto \theta^{\alpha_0 - 1}(1-\theta)^{\beta_0 - 1}$, the posterior is

$$p(\theta \mid k, n) \propto \theta^{\alpha_0 + k - 1}(1-\theta)^{\beta_0 + (n-k) - 1} = \text{Beta}(\alpha_0 + k,\ \beta_0 + n - k).$$

Plugging in $\alpha_0 = 2$, $\beta_0 = 2$, $k = 7$, $n = 10$:

$$\boxed{\theta \mid k, n \sim \text{Beta}(2 + 7,\ 2 + 3) = \text{Beta}(9, 5).}$$

From this we read off three key summaries:

- **Prior mean** $= \alpha_0/(\alpha_0 + \beta_0) = 2/4 = 0.50$.
- **MLE** $= k/n = 7/10 = 0.70$.
- **Posterior mean** $= 9/14 \approx 0.643$.

Why this matters. Notice that the prior acts as if you had already seen $\alpha_0 - 1 = 1$ head and $\beta_0 - 1 = 1$ tail ("pseudo-counts"). The posterior simply adds the real observations to these pseudo-counts. The more informative the prior (larger $\alpha_0 + \beta_0$), the more it resists being moved by the data.

(b) The posterior mean as a precision-weighted compromise.
For the Beta–Binomial model, :

$$\mu_N \approx 0.643.$$

Because the posterior mean is a *weighted average* with positive weights, it must lie strictly between the two endpoints:

$$0.50 < 0.643 < 0.70.$$

This phenomenon is called **shrinkage**: the posterior "shrinks" the data-driven estimate toward the prior. The direction and magnitude of shrinkage depend on how informative the prior is relative to the data.

(c) The large-n limit and epistemic uncertainty.
The posterior variance satisfies

$$\text{Var}[\theta \mid \text{data}] = \frac{(\alpha_0 + k)(\beta_0 + n - k)}{(\alpha_0 + \beta_0 + n)^2(\alpha_0 + \beta_0 + n + 1)} = O(1/n) \xrightarrow{n\to\infty} 0.$$

The posterior concentrates on the true value of θ: it becomes a spike (delta function) at θ_{true}.
Interpretation in terms of uncertainty types:

- **Epistemic uncertainty** (uncertainty about θ) is captured by the posterior variance. It is *reducible*: more data shrinks it to zero.
- **Aleatoric uncertainty** (the inherent randomness of each coin flip) remains $\theta_{\text{true}}(1-\theta_{\text{true}})$ no matter how much data we collect — it is *irreducible*.

This is the defining distinction of the course: epistemic uncertainty reflects our ignorance and vanishes with data; aleatoric uncertainty is a property of the data-generating process and persists forever.
Example. After $n = 10$ flips, $\text{Var}[\theta \mid \text{data}] = 9 \times 5/(14^2 \times 15) \approx 0.0153$. After $n = 1000$ flips with the same head rate, the variance would drop to approximately 0.0002: we know θ to within ± 0.015. But each individual flip is still random.

2 Topic 2: Gaussian Models & Bayesian Linear Regression

R2: Predictive Uncertainty Decomposition

In Bayesian linear regression: $p(y_* \mid \mathbf{x}_*, \mathcal{D}) = \mathcal{N}(\mathbf{x}_*^\top \boldsymbol{\mu}_N,\ \sigma^2 + \mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*)$.
(a) Identify aleatoric and epistemic terms.
(b) Show the posterior precision satisfies $\boldsymbol{\Sigma}_N^{-1} = \boldsymbol{\Sigma}_0^{-1} + \sigma^{-2}\mathbf{X}^\top\mathbf{X}$.
(c) How does each uncertainty component behave as $N \to \infty$ and as $\mathbf{x}_*$ moves far from training data?

Solution

(a) Revision: the two sources of uncertainty.
The predictive variance in Bayesian linear regression is

$$\text{Var}[y_* \mid \mathbf{x}_*, \mathcal{D}] = \underbrace{\sigma^2}_{\text{aleatoric}} + \underbrace{\mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*}_{\text{epistemic}}.$$

Why these names?

- σ^2 is the **observation noise variance**. It represents randomness inherent in the data-generating process $y = \mathbf{x}^\top\mathbf{w} + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$. No amount of data can eliminate it. This is **aleatoric** (irreducible) uncertainty.
- $\mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*$ measures how uncertain we are about the *weights* $\mathbf{w}$. $\boldsymbol{\Sigma}_N$ is the posterior covariance of $\mathbf{w}$, and the quadratic form projects this weight uncertainty into the prediction

direction $\mathbf{x}_*$. This is **epistemic** (reducible) uncertainty: with more data, $\boldsymbol{\Sigma}_N$ shrinks and so does this term.

Numerical example. Suppose $\sigma^2 = 1$, and for a particular test point $\mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_* = 0.25$. Then the total predictive variance is 1.25, and the predictive standard deviation is $\sqrt{1.25} \approx 1.12$. Of this, $1/1.25 = 80\%$ comes from observation noise and $0.25/1.25 = 20\%$ from parameter uncertainty.

(b) Derivation of the posterior precision.

We start from Bayes' rule in log-space:

$$\log p(\mathbf{w} \mid \mathcal{D}) = \log p(\mathcal{D} \mid \mathbf{w}) + \log p(\mathbf{w}) + \text{const.}$$

The prior is $\mathbf{w} \sim \mathcal{N}(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$, so

$$\log p(\mathbf{w}) = -\tfrac{1}{2}(\mathbf{w} - \boldsymbol{\mu}_0)^\top \boldsymbol{\Sigma}_0^{-1}(\mathbf{w} - \boldsymbol{\mu}_0) + \text{const.}$$

The likelihood for N i.i.d. observations $\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$, is

$$\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = -\frac{1}{2\sigma^2}(\mathbf{y} - \mathbf{X}\mathbf{w})^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) + \text{const.}$$

Now collect all terms that are quadratic in $\mathbf{w}$:

$$-\tfrac{1}{2}\mathbf{w}^\top \boldsymbol{\Sigma}_0^{-1}\mathbf{w} - \tfrac{1}{2\sigma^2}\mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} = -\tfrac{1}{2}\mathbf{w}^\top \underbrace{\left(\boldsymbol{\Sigma}_0^{-1} + \sigma^{-2}\mathbf{X}^\top\mathbf{X}\right)}_{\boldsymbol{\Sigma}_N^{-1}}\mathbf{w}.$$

Since the log-posterior is quadratic in $\mathbf{w}$, the posterior is Gaussian, and the coefficient of the quadratic form is the posterior precision:

$$\boxed{\boldsymbol{\Sigma}_N^{-1} = \boldsymbol{\Sigma}_0^{-1} + \sigma^{-2}\mathbf{X}^\top\mathbf{X}.}$$

The "precisions add" principle. This is one of the most fundamental results in the course. Each data point $(\mathbf{x}_i, y_i)$ contributes $\sigma^{-2}\mathbf{x}_i\mathbf{x}_i^\top$ to the posterior precision matrix. More data $\Rightarrow$ higher precision $\Rightarrow$ lower uncertainty. The prior precision $\boldsymbol{\Sigma}_0^{-1}$ plays the role of "pseudo-data". This same principle recurs in the Kalman filter (Topic 4) and sensor fusion (Topic 8).

1D example. If $x \in \mathbb{R}$, prior precision $\lambda_0 = 1/\sigma_0^2$, then $\lambda_N = \lambda_0 + N/\sigma^2$. With $\sigma_0 = 2$, $\sigma = 1$, $N = 10$: $\lambda_N = 0.25 + 10 = 10.25$, so $\sigma_N^2 = 1/10.25 \approx 0.098$. Each observation adds one "unit" of precision $1/\sigma^2 = 1$.

(c) Behaviour of the two components.

As $N \to \infty$: The data precision $\sigma^{-2}\mathbf{X}^\top\mathbf{X}$ grows (assuming $\frac{1}{N}\mathbf{X}^\top\mathbf{X} \to \boldsymbol{\Sigma}_x$ finite and positive-definite). Therefore $\boldsymbol{\Sigma}_N = (\boldsymbol{\Sigma}_0^{-1} + \sigma^{-2}\mathbf{X}^\top\mathbf{X})^{-1} \to \mathbf{0}$, and the epistemic term $\mathbf{x}_*^\top\boldsymbol{\Sigma}_N\mathbf{x}_* \to 0$. The aleatoric term σ^2 is completely unaffected: it is a property of the noise, not of our knowledge. In the infinite-data limit, the total predictive variance converges to σ^2, recovering the frequentist result.

As $\mathbf{x}_*$ moves far from training data: The quadratic form $\mathbf{x}_*^\top\boldsymbol{\Sigma}_N\mathbf{x}_*$ grows because $\boldsymbol{\Sigma}_N$ has not been "shrunk" in directions not covered by the training inputs. Intuitively, if the model has never seen data near $\mathbf{x}_*$, it has high epistemic uncertainty there. Meanwhile, the aleatoric term σ^2 remains constant — observation noise does not depend on where you predict.

The model "knows it doesn't know." This is the hallmark of a properly Bayesian model: uncertainty fans out in regions of sparse data. A point estimate (OLS) gives the same confidence everywhere and misses this crucial signal.

3 Topic 3: MCMC, Langevin Dynamics, & Logistic Regression

R3: From Random Walk to Gradient-Informed Sampling

(a) Write the MH acceptance ratio for a symmetric random-walk proposal. Show $p(\mathbf{x})$ cancels.
(b) ULA adds gradient information: $\theta^* = \theta + \frac{\Delta t}{2}\nabla \log \pi(\theta) + \sqrt{\Delta t}\,\mathbf{z}$. Is this proposal symmetric? What is the consequence?
(c) How does MALA fix ULA? Why does the Laplace approximation (logistic regression lecture) provide a good MCMC initialisation?

Solution

(a) Revision: the Metropolis–Hastings algorithm.
The MH algorithm generates a Markov chain whose stationary distribution is the target $\pi(\theta) = p(\theta \mid \text{data})$. At each step we: (1) propose $\theta^* \sim q(\theta^* \mid \theta)$, (2) accept with probability $r = \min(1, A)$ where

$$A = \frac{\pi(\theta^*)\, q(\theta \mid \theta^*)}{\pi(\theta)\, q(\theta^* \mid \theta)}.$$

For a **symmetric** random-walk proposal, $q(\theta^* \mid \theta) = q(\theta \mid \theta^*)$ (e.g., $\theta^* = \theta + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma_p^2)$), so the proposal ratio cancels:

$$A = \frac{\pi(\theta^*)}{\pi(\theta)}.$$

Now, $\pi(\theta) = \tilde{\pi}(\theta)/Z$ where $Z = \int \tilde{\pi}(\theta)\, d\theta$ is the normalising constant and $\tilde{\pi}(\theta) = p(\text{data} \mid \theta)\, p(\theta)$ is the unnormalised posterior. Since Z appears in both numerator and denominator:

$$A = \frac{\tilde{\pi}(\theta^*)/Z}{\tilde{\pi}(\theta)/Z} = \frac{\tilde{\pi}(\theta^*)}{\tilde{\pi}(\theta)}.$$

Why this matters. We *never need to compute* Z. This is the key power of MCMC: it lets us sample from distributions we can only evaluate up to a constant. In Bayesian inference, $Z = p(\text{data})$ (the marginal likelihood / evidence) is typically intractable — but MCMC bypasses it entirely.
Example. Suppose $\tilde{\pi}(\theta) = e^{-\theta^2/2}$ (unnormalised standard normal). If $\theta = 0.5$ and $\theta^* = 1.2$: $A = e^{-1.44/2}/e^{-0.25/2} = e^{-0.595} \approx 0.55$. We accept the move with probability 55%. The chain spends more time near $\theta = 0$ (the mode) but still explores the tails.

(b) ULA: gradient-informed proposals and their bias.
The **Unadjusted Langevin Algorithm** (ULA) replaces the blind random walk with a proposal that "knows" where the high-density region is:

$$\theta^* = \theta + \frac{\Delta t}{2}\nabla_\theta \log \pi(\theta) + \sqrt{\Delta t}\,\mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

The term $\frac{\Delta t}{2}\nabla \log \pi(\theta)$ is the **drift**: it pushes the proposal toward regions of high posterior density (uphill on $\log \pi$). The noise $\sqrt{\Delta t}\,\mathbf{z}$ ensures exploration.
Is this proposal symmetric? No. The proposal $q(\theta^* \mid \theta)$ is centred at $\theta + \frac{\Delta t}{2}\nabla \log \pi(\theta)$, while the reverse proposal $q(\theta \mid \theta^*)$ is centred at $\theta^* + \frac{\Delta t}{2}\nabla \log \pi(\theta^*)$. Since $\nabla \log \pi(\theta) \neq \nabla \log \pi(\theta^*)$ in general, we have $q(\theta^* \mid \theta) \neq q(\theta \mid \theta^*)$.

Consequence. Because ULA uses these proposals *without* an accept/reject correction, its Markov chain does not have $\pi(\theta)$ as its exact stationary distribution. The stationary distribution is biased — it is only correct in the limit $\Delta t \to 0$. For any finite step size, ULA samples from a distribution that is close to, but not exactly, $\pi(\theta)$. Larger Δt means faster exploration but worse bias.

Intuition. Think of ULA as a ball rolling downhill on the log-posterior landscape with random kicks. Because the ball takes discrete steps, it overshoots curves and accumulates error. Only infinitesimally small steps (continuous-time limit) give the exact dynamics.

(c) MALA and the Laplace initialisation.

MALA (Metropolis-Adjusted Langevin Algorithm) takes ULA's gradient-informed proposal and wraps it in a standard MH accept/reject step. The acceptance ratio now includes the asymmetric proposal densities:

$$A = \frac{\pi(\theta^*)\, q(\theta \mid \theta^*)}{\pi(\theta)\, q(\theta^* \mid \theta)},$$

where $q(\theta^* \mid \theta) = \mathcal{N}\big(\theta + \frac{\Delta t}{2}\nabla \log \pi(\theta),\ \Delta t\, \mathbf{I}\big)$. The accept/reject step exactly corrects for the discretisation error, so MALA's stationary distribution is *exactly* $\pi(\theta)$ for *any* step size Δt. The cost is that some proposals are rejected, but the benefit is unbiased samples.

Summary of the hierarchy: Random walk MH uses no gradient information (slow but exact); ULA uses gradient information but is biased; MALA uses gradient information *and* accept/reject correction (fast and exact).

Why the Laplace approximation helps. The Laplace approximation (from the logistic regression lecture) finds $\hat{\mathbf{w}}_{\text{MAP}} = \arg\max_{\mathbf{w}} \log p(\mathbf{w} \mid \mathcal{D})$ and approximates the posterior as $\mathcal{N}(\hat{\mathbf{w}}_{\text{MAP}}, \mathbf{H}^{-1})$ where $\mathbf{H} = -\nabla^2 \log p(\mathbf{w} \mid \mathcal{D})\big|_{\hat{\mathbf{w}}}$ is the Hessian at the mode. If we initialise an MCMC chain at $\hat{\mathbf{w}}_{\text{MAP}}$, the chain starts in the high-density region of the posterior. This drastically reduces **burn-in** — the number of samples discarded before the chain reaches stationarity. Without a good initialisation, the chain might wander in low-density regions for thousands of steps.

Example. For logistic regression with 100 features, finding the MAP takes a few seconds of Newton optimisation. Starting MALA from the MAP, the chain mixes almost immediately. Starting from a random point, the chain might need 10,000+ steps to find the posterior mass.

4 Topic 4: State Space Models, Kalman & Particle Filters

R4: From Kalman to Particles: When Gaussianity Breaks

(a) For $z_{t+1} = z_t + q_t$, $y_t = z_t + r_t$, $Q = 1$, $R = 4$, $\hat{z}_{0|0} = 5$, $P_{0|0} = 2$: compute predict and update after $y_1 = 11$.

(b) Name two situations where the Kalman filter fails and the particle filter is needed.

(c) Define weight degeneracy in the particle filter and explain how resampling addresses it.

Solution

(a) Revision: the Kalman filter predict–update cycle.
The Kalman filter operates on a linear-Gaussian state space model. Here $F = 1$ (state transition), $H = 1$ (observation matrix), $Q = 1$ (process noise variance), $R = 4$ (observation noise variance). The two steps are:
Predict step (propagate forward in time using the dynamics):

$$\hat{z}_{1|0} = F\,\hat{z}_{0|0} = 1 \times 5 = 5,$$
$$P_{1|0} = F\,P_{0|0}\,F^{\top} + Q = 1 \times 2 \times 1 + 1 = 3.$$

Interpretation: before seeing y_1, our best guess is still 5, but uncertainty has grown from 2 to 3 because the process noise $Q = 1$ added to our uncertainty.
Update step (incorporate the measurement $y_1 = 11$):
First, compute the **Kalman gain**:

$$K_1 = \frac{P_{1|0}\,H^{\top}}{H\,P_{1|0}\,H^{\top} + R} = \frac{3 \times 1}{1 \times 3 \times 1 + 4} = \frac{3}{7} \approx 0.429.$$

The Kalman gain tells us how much to trust the measurement relative to the prediction. Since $R = 4 > P_{1|0} = 3$, the measurement is noisier than the prediction, so $K < 0.5$: we trust the prediction slightly more.
Next, compute the **innovation** (surprise):

$$\nu_1 = y_1 - H\,\hat{z}_{1|0} = 11 - 5 = 6.$$

The measurement is 6 units above our prediction — a large surprise.
Updated state estimate:

$$\hat{z}_{1|1} = \hat{z}_{1|0} + K_1\,\nu_1 = 5 + \frac{3}{7} \times 6 = 5 + \frac{18}{7} \approx 7.57.$$

We compromise: the prediction said 5, the measurement said 11, and we land at 7.57 — weighted toward the prediction because we trust it more.
Updated covariance:

$$P_{1|1} = (1 - K_1\,H)\,P_{1|0} = \left(1 - \tfrac{3}{7}\right) \times 3 = \frac{4}{7} \times 3 = \frac{12}{7} \approx 1.71.$$

The posterior uncertainty (1.71) is smaller than both the predicted uncertainty (3) and the measurement uncertainty (4). This is "precisions add": $P_{1|1}^{-1} = P_{1|0}^{-1} + R^{-1} = 1/3 + 1/4 = 7/12$, so $P_{1|1} = 12/7$.
Sanity check. We can verify via the precision form: $1/P_{1|1} = 1/3 + 1/4 = 7/12$, giving $P_{1|1} = 12/7 \approx 1.71$. ✓

(b) When the Kalman filter fails.
The Kalman filter is *optimal* only when: (i) the dynamics and observation models are linear, and (ii) all noise is Gaussian. Two common situations that violate these assumptions:
1. Nonlinear observation model. Example: bearings-only tracking, where a sensor measures $y = \arctan(z_y/z_x) + r$. The arctan function is nonlinear in the state, so the posterior is not Gaussian. The EKF (extended KF) linearises this, but can diverge if the nonlinearity is

strong. A particle filter represents the posterior with weighted samples and handles arbitrary nonlinearities.

2. Multimodal posterior. Example: a target could be in one of two corridors, and the measurement is ambiguous (data association uncertainty). The true posterior has two peaks. A single Gaussian (Kalman filter) cannot represent two peaks — it would average them, placing the estimate in a wall between the corridors. A particle filter naturally represents multimodality by placing particles in both modes.

Common pitfall. Students sometimes say "the particle filter is always better." It is more general, but for high-dimensional linear-Gaussian systems, the Kalman filter is exact and vastly cheaper. Particle filters suffer from the curse of dimensionality.

(c) Weight degeneracy and resampling.

The problem. In a particle filter, we represent the posterior with N weighted particles $\{(\mathbf{z}^{(i)}, w^{(i)})\}_{i=1}^{N}$. After several predict–update cycles, the weights become highly unequal: typically one particle has $w^{(i)} \approx 1$ and the rest have $w^{(j)} \approx 0$. This is **weight degeneracy**.

We quantify it with the **effective sample size**:

$$N_{\text{eff}} = \frac{1}{\sum_{i=1}^{N} (w^{(i)})^2}.$$

When all weights are equal ($w^{(i)} = 1/N$), $N_{\text{eff}} = N$ (full diversity). When one particle dominates, $N_{\text{eff}} \approx 1$ (complete degeneracy).

Why it happens. At each time step, weights are multiplied by the likelihood $p(y_t \mid \mathbf{z}_t^{(i)})$. Particles that happen to be near the observation receive high likelihood factors; others receive near-zero factors. Over multiple steps, these multiplicative updates compound, causing exponential divergence.

Numerical example. Start with 4 equal particles. After one step: weights $[0.5, 0.3, 0.15, 0.05]$, $N_{\text{eff}} \approx 2.7$. After three more steps: weights might be $[0.98, 0.015, 0.004, 0.001]$, $N_{\text{eff}} \approx 1.04$. Nearly all computational effort is wasted on particles with negligible weight.

The solution: resampling. When N_{eff} drops below a threshold (typically $N/2$), we **resample**: draw N new particles with replacement from the current set, with probability proportional to the weights. High-weight particles are duplicated; low-weight particles are discarded. All weights are then reset to $1/N$.

Trade-off. Resampling concentrates particles in high-probability regions (good), but introduces **sample impoverishment**: many particles become identical copies, reducing diversity. Strategies like systematic resampling, residual resampling, and adding jitter (regularised PF) mitigate this.

5 Topic 5: Variational Inference

R5: The ELBO: Derivation, Decomposition, and Reparameterization

(a) Derive the ELBO from $\log p(\mathbf{x})$ using Jensen's inequality.
(b) Show $\mathcal{L}(q) = \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})] - D_{\mathrm{KL}}(q(\mathbf{z}) \| p(\mathbf{z}))$. Interpret.
(c) What is the reparameterization trick and why is it necessary?
(d) Compare forward KL (mode-covering) vs. reverse KL (mode-seeking). Which does VI minimise?

Solution

(a) Derivation of the ELBO from Jensen's inequality.
We start from the log marginal likelihood (also called the *evidence*):

$$\log p(\mathbf{x}) = \log \int p(\mathbf{x}, \mathbf{z})\, d\mathbf{z}.$$

This integral is typically intractable because $\mathbf{z}$ is high-dimensional and the integrand is complex. We introduce a *variational distribution* $q(\mathbf{z})$ and use it to rewrite the integral:

$$\log p(\mathbf{x}) = \log \int q(\mathbf{z}) \frac{p(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})}\, d\mathbf{z} = \log \mathbb{E}_q\left[\frac{p(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})}\right].$$

Since log is concave, Jensen's inequality gives $\log \mathbb{E}[\cdot] \geq \mathbb{E}[\log(\cdot)]$:

$$\log p(\mathbf{x}) \geq \mathbb{E}_q\left[\log \frac{p(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})}\right] = \int q(\mathbf{z}) \log \frac{p(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})}\, d\mathbf{z} \;\equiv\; \mathcal{L}(q).$$

This lower bound $\mathcal{L}(q)$ is the **Evidence Lower BOund (ELBO)**.
When is equality achieved? Jensen's inequality is tight when the random variable inside the expectation is constant, i.e., $p(\mathbf{x}, \mathbf{z})/q(\mathbf{z}) = \text{const}$. This happens if and only if $q(\mathbf{z}) = p(\mathbf{z} \mid \mathbf{x})$, the true posterior. Equivalently, the gap is $\log p(\mathbf{x}) - \mathcal{L}(q) = D_{\mathrm{KL}}(q(\mathbf{z}) \| p(\mathbf{z} \mid \mathbf{x})) \geq 0$.
Why maximise the ELBO? Since $\log p(\mathbf{x})$ is fixed, maximising $\mathcal{L}(q)$ is equivalent to minimising $D_{\mathrm{KL}}(q \| p(\mathbf{z} \mid \mathbf{x}))$, i.e., making q as close as possible to the true posterior.

(b) Decomposition and interpretation.
We expand the joint:

$$\begin{aligned}
\mathcal{L}(q) &= \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z})] - \mathbb{E}_q[\log q(\mathbf{z})] \\
&= \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z}) + \log p(\mathbf{z})] - \mathbb{E}_q[\log q(\mathbf{z})] \\
&= \underbrace{\mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})]}_{\text{reconstruction / data fit}} - \underbrace{D_{\mathrm{KL}}(q(\mathbf{z}) \| p(\mathbf{z}))}_{\text{complexity / regularisation}}.
\end{aligned}$$

Interpretation of each term:

- **Reconstruction term** $\mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})]$: measures how well the latent codes $\mathbf{z}$ sampled from q explain the observed data $\mathbf{x}$. Maximising this alone would make q concentrate on the single $\mathbf{z}$ that best reconstructs $\mathbf{x}$ (overfitting to a point estimate).
- **KL term** $D_{\mathrm{KL}}(q(\mathbf{z}) \| p(\mathbf{z}))$: penalises q for deviating from the prior $p(\mathbf{z})$. This prevents q from becoming a delta function and encourages it to stay close to a sensible default (e.g., $\mathcal{N}(\mathbf{0}, \mathbf{I})$).

This is the Bayesian **data-fit vs. complexity trade-off**: the model must explain the data but also remain simple (close to the prior). In the VAE (Topic 6), this decomposition directly governs the loss function.

Example. If q concentrates on $\mathbf{z} = \mathbf{z}_{\text{MAP}}$, the reconstruction is excellent but the KL is huge (q is far from the prior). If $q = p(\mathbf{z})$, the KL is zero but reconstruction is poor (we are ignoring the data). The ELBO optimum balances these.

(c) The reparameterization trick.

The problem. To optimise the ELBO with respect to the parameters ϕ of $q_\phi(\mathbf{z})$, we need $\nabla_\phi \mathcal{L}$. The difficulty is that the expectation $\mathbb{E}_{q_\phi}[\cdot]$ is taken with respect to a distribution that itself depends on ϕ:

$$\nabla_\phi \, \mathbb{E}_{q_\phi(\mathbf{z})}[f(\mathbf{z})] = \nabla_\phi \int q_\phi(\mathbf{z}) \, f(\mathbf{z}) \, d\mathbf{z}.$$

We cannot simply move ∇_ϕ inside the integral because q_ϕ depends on ϕ.

The trick. Reparameterise: instead of sampling $\mathbf{z} \sim q_\phi = \mathcal{N}(\boldsymbol{\mu}_\phi, \text{diag}(\boldsymbol{\sigma}_\phi^2))$, write

$$\mathbf{z} = \boldsymbol{\mu}_\phi + \boldsymbol{\sigma}_\phi \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

Now $\mathbf{z}$ is a *deterministic, differentiable* function of ϕ (through $\boldsymbol{\mu}_\phi$ and $\boldsymbol{\sigma}_\phi$) plus noise $\boldsymbol{\epsilon}$ that does not depend on ϕ. The gradient becomes

$$\nabla_\phi \, \mathbb{E}_{\boldsymbol{\epsilon}}[f(\boldsymbol{\mu}_\phi + \boldsymbol{\sigma}_\phi \odot \boldsymbol{\epsilon})] = \mathbb{E}_{\boldsymbol{\epsilon}}[\nabla_\phi \, f(\boldsymbol{\mu}_\phi + \boldsymbol{\sigma}_\phi \odot \boldsymbol{\epsilon})],$$

which we can estimate with a single Monte Carlo sample of $\boldsymbol{\epsilon}$ and backpropagation.

Why it is necessary. Without the trick, we would need high-variance score-function estimators (REINFORCE), which make training unstable. The reparameterization trick gives low-variance gradient estimates, enabling efficient SGD training of VAEs, BNNs (Bayes by Backprop), and SVI.

(d) Forward vs. reverse KL.

There are two ways to measure how close q is to the true posterior $p \equiv p(\mathbf{z} \mid \mathbf{x})$:

- **Forward KL**: $D_{\text{KL}}(p\|q) = \mathbb{E}_p[\log(p/q)]$. This is **mode-covering** (also called "mean-seeking"): q must be nonzero wherever p is nonzero, so q tends to spread out to cover all modes of p, even at the cost of placing mass in low-density regions.
- **Reverse KL**: $D_{\text{KL}}(q\|p) = \mathbb{E}_q[\log(q/p)]$. This is **mode-seeking** (also called "zero-forcing"): q avoids placing mass where $p \approx 0$ (because $\log(q/p) \to \infty$ there). Instead, q concentrates on one mode and may entirely miss others.

VI minimises the reverse KL $D_{\text{KL}}(q\|p)$. This is a direct consequence of the ELBO derivation: $\log p(\mathbf{x}) - \mathcal{L}(q) = D_{\text{KL}}(q\|p)$, and maximising $\mathcal{L}$ is equivalent to minimising $D_{\text{KL}}(q\|p)$.

Practical implication. If the true posterior is multimodal (e.g., two well-separated peaks), a factored Gaussian q will lock onto one mode and ignore the other. This is acceptable when we only need a good local approximation, but can be problematic when all modes are important.

6 Topic 6: NN Parameterization, VAEs, & BNNs

R6: Three Neural Approaches to Uncertainty

For each method, state what is random at test time, what uncertainty type it captures, and its main limitation: (a) Heteroscedastic network, (b) MC Dropout, (c) Bayes by Backprop.

Solution

Revision: why do we need neural approaches to uncertainty?
In classical Bayesian linear regression (Topic 2), the posterior $p(\mathbf{w} \mid \mathcal{D})$ is available in closed form. For neural networks, the posterior over millions of weights is intractable. Topics 6 introduces three practical strategies, each with different trade-offs.

	Heteroscedastic	**MC Dropout**	**Bayes by Backprop**
Random at test	Nothing	Dropout masks	Weights $\mathbf{w} \sim q_\phi$
Passes	1	T	T
Uncertainty	Aleatoric	Epistemic	Epistemic
Limitation	No epistemic	Restricted q	$2\times$ params

(a) Heteroscedastic network.
Key idea. A standard regression network outputs only a point prediction $\hat{y}(\mathbf{x})$. A heteroscedastic network outputs *two* quantities: a mean $\mu(\mathbf{x})$ and an input-dependent variance $\sigma^2(\mathbf{x})$. The loss is the negative log-likelihood of a Gaussian:

$$\mathcal{L} = \frac{1}{2}\sum_i \left[\frac{(y_i - \mu(\mathbf{x}_i))^2}{\sigma^2(\mathbf{x}_i)} + \log \sigma^2(\mathbf{x}_i)\right].$$

The first term penalises large residuals (data fit); the second term penalises overly large σ^2 (the network cannot "cheat" by predicting infinite variance to make residuals costless).
What is random at test time? Nothing — the network is deterministic. Given $\mathbf{x}_*$, it outputs a single (μ, σ^2) pair.
What uncertainty does it capture? Only **aleatoric** uncertainty: input-dependent noise. For example, near a noisy sensor the predicted $\sigma^2(\mathbf{x})$ will be large.
Limitation. Because the weights are fixed (point estimates), the model has no way to express "I have never seen data like $\mathbf{x}_*$". It produces confident predictions even far from the training distribution. It captures *no epistemic uncertainty.*

(b) MC Dropout.
Key idea. Dropout is normally a regulariser used only during training. Gal & Ghahramani (2016) showed that keeping dropout active at test time and running T forward passes approximates Bayesian inference. Each pass uses a different random binary mask, producing a different prediction $\hat{y}_t(\mathbf{x}_*)$.
What is random at test time? The dropout masks — each forward pass randomly zeros out a different subset of neurons.
What uncertainty does it capture? Primarily **epistemic** uncertainty. The variance across the T predictions, $\frac{1}{T}\sum_t(\hat{y}_t - \bar{y})^2$, reflects disagreement caused by weight uncertainty. Far from training data, predictions diverge more, producing higher variance.

Limitation. The implicit variational family q is a product of Bernoulli distributions (dropout masks) times the point-estimate weights. This is a restricted family that cannot capture complex posterior correlations between weights. Also, the dropout rate p is a hyperparameter that conflates regularisation strength and approximation quality.

(c) Bayes by Backprop (BbB).
Key idea. BbB directly parameterises a distribution over every weight: $w_j \sim q_\phi(w_j) = \mathcal{N}(\mu_j, \sigma_j^2)$. Training optimises the ELBO (from Topic 5) using the reparameterization trick. At test time, we draw T weight samples and average predictions.
What is random at test time? The weights themselves are sampled: $\mathbf{w}^{(t)} \sim \prod_j \mathcal{N}(\mu_j, \sigma_j^2)$.
What uncertainty does it capture? Epistemic uncertainty via weight posterior. If σ_j is large, the network is uncertain about weight j; predictions will vary across samples.
Limitation. We need $2\times$ parameters (μ_j and σ_j for each weight), doubling memory. The mean-field factorisation $q(\mathbf{w}) = \prod_j q(w_j)$ ignores correlations between weights. Training can be challenging due to the high variance of the ELBO gradient.

Combining methods for full uncertainty.
A powerful strategy is to combine a heteroscedastic output head (for aleatoric) with MC Dropout or BbB (for epistemic). The total predictive variance then decomposes as:

$$\mathrm{Var}[y_*] \approx \underbrace{\frac{1}{T}\sum_t \sigma_t^2(\mathbf{x}_*)}_{\text{mean aleatoric}} + \underbrace{\frac{1}{T}\sum_t (\mu_t(\mathbf{x}_*) - \bar{\mu})^2}_{\text{epistemic (disagreement)}}.$$

This mirrors the decomposition from Bayesian linear regression (Topic 2), now applicable to deep networks.
Connection to VAEs. The VAE is another instance of the ELBO framework (amortised VI): the encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ plays the role of the variational distribution, and the decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$ plays the role of the likelihood. BbB and VAE both use the reparameterization trick from Topic 5, but for different latent variables (weights vs. latent codes).

7 Topic 7: Calibration, Scoring Rules, & Conformal Prediction

R7: Calibration, Temperature Scaling, and Conformal Guarantees

(a) A reliability diagram shows all bins above the diagonal. Under- or over-confident?
(b) Should temperature T be > 1 or < 1 to fix it?
(c) State the split conformal recipe for regression and its finite-sample guarantee.
(d) Why is conformal prediction complementary to (not a replacement for) calibration?

Solution

(a) Revision: reliability diagrams and calibration.
A **reliability diagram** groups predictions into bins by predicted confidence and plots observed accuracy (y-axis) against predicted confidence (x-axis). A perfectly calibrated model lies on the diagonal $y = x$: "when I say 70%, I am right 70% of the time."

All bins above the diagonal means: in every bin, the observed accuracy exceeds the predicted confidence.

Concrete example. Suppose in the bin with average predicted confidence 0.60, the empirical accuracy is 0.75. The model says "I am 60% sure," but reality says "you are right 75% of the time." The model is *too pessimistic* about its own abilities.

This is **under-confidence**: the model's probabilities are systematically too low. (Over-confidence would be the opposite: bins below the diagonal, model claims higher confidence than it achieves.)

Common confusion. Students sometimes mix up the direction. A mnemonic: "above the diagonal = the model is *better* than it thinks = **under**-confident."

(b) Revision: temperature scaling.

Temperature scaling modifies the softmax probabilities by dividing the logits $\mathbf{z}$ by a scalar $T > 0$ before applying softmax:

$$p_T(y = c \mid \mathbf{x}) = \frac{e^{z_c/T}}{\sum_j e^{z_j/T}}.$$

The effect of T:

- $T = 1$: original model output.
- $T > 1$: divides logits by a number > 1, making them closer to zero $\Rightarrow$ softmax output flattens $\Rightarrow$ less confident.
- $T < 1$: divides logits by a number < 1 (equivalently, multiplies logits), making differences larger $\Rightarrow$ softmax output becomes peakier $\Rightarrow$ more confident.

Since the model is under-confident (from part (a)), we need to *increase* its confidence. Therefore $\boxed{T < 1}$.

Numerical example. Logits: [2.0, 1.0, 0.0]. At $T = 1$: softmax $\approx$ [0.67, 0.24, 0.09]. At $T = 0.5$: logits become [4.0, 2.0, 0.0], softmax $\approx$ [0.84, 0.13, 0.03]. The top-class probability increased from 0.67 to 0.84 — more confident, as desired.

Important note. Temperature scaling does not change the *ranking* of classes (the predicted label stays the same) — it only recalibrates the probabilities. It is a post-hoc method: train the model first, then find T by minimising NLL on a validation set.

(c) Split conformal prediction for regression.

Step 1: Train a model. Fit any regression model $\hat{f}(x)$ on a training set. The model can be a linear regression, random forest, neural network, or anything else — conformal prediction is **model-agnostic**.

Step 2: Compute nonconformity scores on a held-out calibration set. For each calibration point (x_i, y_i), compute the absolute residual:

$$s_i = |y_i - \hat{f}(x_i)|.$$

This measures "how wrong was my model on this example."

Step 3: Find the calibration quantile. Choose a target miscoverage rate α (e.g., $\alpha = 0.1$ for 90% coverage). Sort the scores and take the $\lceil (1-\alpha)(n+1) \rceil / n$-th quantile $\hat{q}$. The finite-sample correction $(n + 1)$ ensures exact coverage even with a finite calibration set.

Step 4: Form prediction intervals. For a new test input x:

$$C(x) = [\hat{f}(x) - \hat{q},\ \hat{f}(x) + \hat{q}].$$

Finite-sample guarantee. Under the assumption of **exchangeability** (which is implied by i.i.d. data):

$$\Pr(y_{n+1} \in C(x_{n+1})) \geq 1 - \alpha.$$

This guarantee is: (i) finite-sample (not asymptotic), (ii) distribution-free (no Gaussian assumption), (iii) model-agnostic (holds regardless of how bad $\hat{f}$ is), (iv) requires *only* exchangeability (weaker than independence).

Numerical example. Suppose $n = 20$ calibration residuals, sorted: $[0.1, 0.3, 0.4, \ldots, 2.1, 2.8]$, and $\alpha = 0.1$. We need the $\lceil 0.9 \times 21 \rceil = 19$-th smallest residual out of 20. If that is $\hat{q} = 2.1$, then $C(x) = [\hat{f}(x) \pm 2.1]$.

Caveat. This basic recipe gives *uniform-width* intervals: $\hat{q}$ is the same for all test points. If the model has varying accuracy across the input space (heteroscedastic noise), these intervals may be too wide in easy regions and too narrow in hard regions. This motivates the connection to part (d).

(d) Why conformal prediction is complementary to calibration.

This is one of the most important conceptual distinctions in the course.

What calibration provides: Calibration makes the model's own uncertainty estimates correspond to reality. A calibrated model can produce **adaptive, input-dependent** intervals: wider where the data is noisy, narrower where it is clean.

What calibration lacks: No hard coverage guarantee. A model might be well-calibrated on average but still violate the $1 - \alpha$ coverage threshold in finite samples, especially under slight misspecification.

What conformal prediction provides: Conformal gives a **formal, finite-sample coverage guarantee**: $\Pr(y \in C(x)) \geq 1 - \alpha$. This is valid regardless of the model's calibration.

What conformal prediction lacks: Basic conformal (absolute residual scores) produces *marginal* coverage with *uniform-width* intervals. These intervals are not adaptive — they are the same width for an easy prediction and a hard one.

The synergy. Calibrate the model first (e.g., via temperature scaling, Platt scaling, or isotonic regression), then apply conformal prediction using the calibrated uncertainty as the nonconformity score (e.g., $s_i = |y_i - \hat{f}(x_i)|/\hat{\sigma}(x_i)$, a normalised residual). This yields:

- **Valid** coverage (from conformal),
- **Adaptive** interval widths (from calibration): wider where $\hat{\sigma}(x)$ is large, narrower where it is small,
- **Efficient** intervals (shorter on average than uncalibrated conformal).

Analogy. Calibration is like tuning a thermometer so it reads correctly. Conformal prediction is like adding error bars to the reading. A well-tuned thermometer with error bars gives both accurate and reliable readings.

8 Topic 8: Sensor Fusion & Data Association

R8: Precision-Weighted Fusion and Mahalanobis Gating

(a) Fuse $y_1 = 4.8$ ($\sigma_1 = 2$), $y_2 = 5.3$ ($\sigma_2 = 1$). Verify $\sigma_f < \min(\sigma_1, \sigma_2)$.
(b) Why is Mahalanobis distance preferred over Euclidean for gating?
(c) How does JPDA use association probabilities β_{ij} in the Kalman update?

Solution

(a) Revision: precision-weighted fusion of two Gaussian measurements.
When two independent sensors measure the same quantity z with Gaussian noise, the optimal (Bayesian) fused estimate is a precision-weighted average.
The **precision** of a measurement is the inverse of its variance: $\lambda_i = 1/\sigma_i^2$. The fused precision is the sum:

$$\lambda_f = \lambda_1 + \lambda_2 = \frac{1}{4} + \frac{1}{1} = 0.25 + 1.0 = 1.25.$$

The fused variance and standard deviation:

$$\sigma_f^2 = \frac{1}{\lambda_f} = \frac{1}{1.25} = 0.8, \qquad \sigma_f = \sqrt{0.8} \approx 0.894.$$

The fused mean is the precision-weighted average of the measurements:

$$\hat{z}_f = \frac{\lambda_1 y_1 + \lambda_2 y_2}{\lambda_f} = \frac{0.25 \times 4.8 + 1.0 \times 5.3}{1.25} = \frac{1.2 + 5.3}{1.25} = \frac{6.5}{1.25} = 5.2.$$

Verification. We need $\sigma_f < \min(\sigma_1, \sigma_2) = \min(2, 1) = 1$. Indeed $\sigma_f \approx 0.894 < 1$. ✓
Why is the fused uncertainty smaller than either individual sensor? Because each sensor provides independent information. The precisions (information) *add*, so the fused precision is always higher than either individual precision. Algebraically: $\lambda_f = \lambda_1 + \lambda_2 > \max(\lambda_1, \lambda_2)$, so $\sigma_f^2 = 1/\lambda_f < 1/\max(\lambda_1, \lambda_2) = \min(\sigma_1^2, \sigma_2^2)$. Two noisy sensors together are better than the best single sensor.
Note on weights. The more precise sensor ($\sigma_2 = 1$) receives weight $w_2 = \lambda_2/\lambda_f = 1.0/1.25 = 0.80$, while the less precise sensor receives $w_1 = 0.25/1.25 = 0.20$. The fused estimate (5.2) is much closer to $y_2 = 5.3$ than to $y_1 = 4.8$: we trust the more precise sensor more. This is exactly the "precisions add" principle from Topics 1, 2, and 4.

(b) Mahalanobis vs. Euclidean distance for gating.
Gating is the process of deciding whether a measurement $\mathbf{y}$ is "close enough" to a predicted measurement $\hat{\mathbf{y}}$ to be considered as a potential association (i.e., coming from the same target).
Euclidean distance: $D_E = \|\mathbf{y} - \hat{\mathbf{y}}\|$. This treats all directions equally and ignores uncertainty.
Mahalanobis distance: $D_M^2 = (\mathbf{y} - \hat{\mathbf{y}})^\top \mathbf{S}^{-1} (\mathbf{y} - \hat{\mathbf{y}})$, where $\mathbf{S} = \mathbf{H}\mathbf{P}_{t|t-1}\mathbf{H}^\top + \mathbf{R}$ is the innovation covariance. This scales the residual by the inverse of the expected covariance.
Why Mahalanobis is better:

- **Accounts for different uncertainty magnitudes.** If position uncertainty along x is much larger than along y, a 10 m residual in x is less surprising than 10 m in y. Euclidean treats them equally; Mahalanobis does not.
- **Accounts for correlations.** If x and y uncertainties are correlated, the gate should be an ellipse, not a circle.
- **Statistical threshold.** Under Gaussian assumptions, $D_M^2 \sim \chi_k^2$ (where k is the measurement dimension). This gives principled thresholds: e.g., $D_M^2 \leq \chi_{2,0.99}^2 = 9.21$ for a 99% gate in 2D. Euclidean distance has no such statistical interpretation.

Example. In radar tracking, range measurements may have $\sigma_r = 10$ m while azimuth measurements (converted to Cartesian) have $\sigma_\theta = 100$ m at long range. A Euclidean gate of 50 m would be too tight in azimuth and too loose in range. Mahalanobis adapts automatically.

(c) JPDA: Joint Probabilistic Data Association.
The problem. In a cluttered environment, multiple measurements may fall within the gate of a single track. Which measurement belongs to the target?

JPDA's approach. Instead of making a hard decision, JPDA computes **association probabilities** $\beta_{ij} = \Pr(\text{measurement } j \text{ originated from target } i)$ for all feasible associations, including the possibility that none of the measurements belong to the target (β_{i0}).
The combined update. For target i, the Kalman update becomes:

$$\hat{z}_{t|t} = \hat{z}_{t|t-1} + K \sum_j \beta_{ij} \underbrace{(y_j - H\hat{z}_{t|t-1})}_{\nu_j \text{ (innovation for meas. } j)} .$$

The effective innovation is the probability-weighted sum of individual innovations. If $\beta_{i1} = 0.7$ and $\beta_{i2} = 0.3$, the filter uses 70% of the innovation from measurement 1 and 30% from measurement 2.
Extra covariance term. JPDA also adds a "spread of the innovations" term to the updated covariance:

$$P_{t|t} = P^{\text{std}}_{t|t} + K \left[\sum_j \beta_{ij} \nu_j \nu_j^\top - \bar{\nu}\bar{\nu}^\top \right] K^\top,$$

where $\bar{\nu} = \sum_j \beta_{ij}\nu_j$ is the combined innovation. This extra term inflates the uncertainty to account for the ambiguity in data association. If all β mass is on one measurement, the extra term vanishes and we recover the standard Kalman update.

9 Topic 9: Diffusion Models

R9: Score, Noise Prediction, and the DDPM Objective

(a) Define the score $s(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x})$. Why does it avoid the normalising constant?
(b) Show: $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \approx -\sqrt{1-\bar{\alpha}_t}\, \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)$.
(c) How does $\mathcal{L}_{\text{simple}} = \mathbb{E}[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2]$ relate to the ELBO from Topic 5?

Solution

(a) Revision: the score function.
The **score** of a distribution $p(\mathbf{x})$ is the gradient of the log-density with respect to $\mathbf{x}$:

$$s(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x}).$$

It is a vector field: at each point $\mathbf{x}$, the score points in the direction of steepest increase of $\log p$.
Why it avoids the normalising constant. Write $p(\mathbf{x}) = \tilde{p}(\mathbf{x})/Z$ where $\tilde{p}$ is the unnormalised density. Then:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = \nabla_{\mathbf{x}} [\log \tilde{p}(\mathbf{x}) - \log Z] = \nabla_{\mathbf{x}} \log \tilde{p}(\mathbf{x}),$$

because $\nabla_{\mathbf{x}} \log Z = \mathbf{0}$ (Z does not depend on $\mathbf{x}$). This is crucial: computing $Z = \int \tilde{p}(\mathbf{x})\, d\mathbf{x}$ is typically intractable (the same integral that makes Bayesian inference hard), but the score sidesteps it entirely.
Example. For $p(\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$:

$$s(\mathbf{x}) = -\boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}).$$

The score points from $\mathbf{x}$ toward the mean $\boldsymbol{\mu}$, with strength proportional to the precision $\boldsymbol{\Sigma}^{-1}$. Far from the mean, the score is large (strong pull); at the mean, it is zero.

Connection to MCMC. The Langevin update (Topic 3) uses the score as its drift term: $\theta_{k+1} = \theta_k + \frac{\Delta t}{2} s(\theta_k) + \sqrt{\Delta t}\,\mathbf{z}$. The score tells the sampler which direction is "uphill" on the posterior.

(b) Score–noise equivalence.

In DDPM, the forward process adds noise gradually. At time step t, the noisy version of $\mathbf{x}_0$ is:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

where $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ is the cumulative noise schedule. This means $q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\ (1-\bar{\alpha}_t)\mathbf{I})$.

Using the Gaussian score formula from part (a):

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t \mid \mathbf{x}_0) = -\frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0}{1-\bar{\alpha}_t} = -\frac{\sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}}{1-\bar{\alpha}_t} = -\frac{\boldsymbol{\epsilon}}{\sqrt{1-\bar{\alpha}_t}}.$$

Rearranging:

$$\boldsymbol{\epsilon} = -\sqrt{1-\bar{\alpha}_t}\,\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t \mid \mathbf{x}_0).$$

Since the neural network $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ is trained to predict $\boldsymbol{\epsilon}$, we get:

$$\boxed{\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \approx -\sqrt{1-\bar{\alpha}_t}\,\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t).}$$

Interpretation. Predicting the noise $\boldsymbol{\epsilon}$ is mathematically equivalent to estimating the score function $\nabla \log q(\mathbf{x}_t)$ up to a known scaling factor. "Noise prediction = score estimation." This connects the practical DDPM training objective to the theory of score-based generative models.

(c) Relating $\mathcal{L}_{\text{simple}}$ to the ELBO.

DDPM as a hierarchical VAE. A DDPM can be viewed as a VAE with T layers of latent variables $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$. The "encoder" (forward process) is fixed: $q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$. The "decoder" (reverse process) is learned: $p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$.

The ELBO for this hierarchical model, called the **variational lower bound (VLB)**, decomposes as:

$$\mathcal{L}_{\text{VLB}} = \sum_{t=2}^{T} D_{\text{KL}}\big(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \,\big\|\, p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)\big) + \text{boundary terms}.$$

Each KL term compares the true reverse step $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ (which is Gaussian and tractable because we condition on $\mathbf{x}_0$) to the learned reverse step $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$.

Since both distributions are Gaussian with *equal variances* (DDPM fixes the reverse variance to a schedule-dependent constant), the KL reduces to:

$$D_{\text{KL}} \propto \|\boldsymbol{\mu}_q - \boldsymbol{\mu}_\theta\|^2 \propto \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2.$$

Summing over t (with time-dependent weights w_t) gives the full VLB.

$\mathcal{L}_{\text{simple}}$ is obtained by **dropping the time-dependent weights** w_t and sampling t uniformly:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}}\big[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2\big].$$

Empirically, this unweighted version trains better (the original weights over-emphasise low-noise steps).

Summary of the connection: DDPM is a hierarchical VAE → its ELBO decomposes into per-step Gaussian KLs → each KL is a noise prediction MSE → dropping weights gives $\mathcal{L}_{\text{simple}}$. So the simple noise-prediction loss is a reweighted version of the ELBO from variational inference, connecting Topics 5 and 9.

10 Topic 10: Multimodal Learning, Attention, & Transformers

R10: Embeddings, Contrastive Learning, and Bayesian Attention

(a) How is a shared embedding space for camera+radar trained via contrastive loss?
(b) Write the self-attention formula. What are Q, K, V?
(c) What does Bayesian attention capture that deterministic attention misses?

Solution

(a) Revision: contrastive learning for cross-modal embeddings.
Goal. Learn a shared d-dimensional embedding space in which camera and radar observations of the *same* scene are close, while observations from *different* scenes are far apart.
Architecture. Two separate encoder networks: $f_{\text{cam}}(\cdot)$ maps a camera image to $\mathbb{R}^d$, and $f_{\text{rad}}(\cdot)$ maps a radar scan to $\mathbb{R}^d$. These encoders have different architectures (e.g., CNN for images, 1D-CNN or MLP for radar) but map to the *same* vector space.
Contrastive loss (InfoNCE). Given a batch of B matched pairs $\{(\mathbf{c}_i, \mathbf{r}_i)\}_{i=1}^{B}$:

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(\text{sim}(\mathbf{c}_i, \mathbf{r}_i)/\tau)}{\sum_{j=1}^{B} \exp(\text{sim}(\mathbf{c}_i, \mathbf{r}_j)/\tau)},$$

where $\text{sim}(\mathbf{c}, \mathbf{r}) = \mathbf{c}^\top \mathbf{r}/(\|\mathbf{c}\|\|\mathbf{r}\|)$ is cosine similarity and τ is a temperature. The numerator rewards high similarity between matched pairs (positives); the denominator penalises similarity with unmatched pairs (negatives).
Intuition. After training, matched camera and radar embeddings are near each other (cosine similarity ≈ 1), while unmatched ones are orthogonal or opposite. This means we can perform **cross-modal retrieval**: given a camera embedding, find the nearest radar embedding (or vice versa) by nearest-neighbour search in the shared space.
Example. CLIP (Contrastive Language–Image Pre-training) does the same thing for images and text. After training, "a photo of a cat" and an actual cat image have high cosine similarity.
Connection to sensor fusion. Once embeddings are aligned, downstream fusion can simply concatenate or average them. The contrastive loss ensures that the two modalities "speak the same language" before fusion.

(b) Self-attention formula.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

What are Q, K, V? Given input token embeddings $\mathbf{X} \in \mathbb{R}^{n \times d}$:

- $Q = \mathbf{X}W_Q$ (**queries**): "what am I looking for?" Each token broadcasts a query vector describing what information it needs.
- $K = \mathbf{X}W_K$ (**keys**): "what do I contain?" Each token broadcasts a key vector advertising what information it offers.
- $V = \mathbf{X}W_V$ (**values**): "what do I provide?" Each token's actual content, to be weighted and aggregated.

Step-by-step:

1. Compute pairwise relevance scores: $S = QK^\top \in \mathbb{R}^{n \times n}$. Entry S_{ij} measures how much token i's query matches token j's key.

2. Scale by $\sqrt{d_k}$ to prevent softmax saturation (without scaling, large d_k makes dot products large, pushing softmax toward one-hot vectors and killing gradients).
3. Apply softmax row-wise: $A = \text{softmax}(S/\sqrt{d_k})$. Row i of A is a probability distribution over all tokens, representing how much attention token i pays to each other token.
4. Weighted aggregation: output $= AV$. Each token's output is a weighted combination of all value vectors, with weights given by the attention distribution.

Numerical example. With 3 tokens and $d_k = 2$: if $Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$, $K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0.5 & 0.5 \end{pmatrix}$, then $QK^\top = \begin{pmatrix} 1 & 0 & 0.5 \\ 0 & 1 & 0.5 \\ 1 & 1 & 1 \end{pmatrix}$. After scaling and softmax, token 3 attends roughly equally to all tokens (its query $[1, 1]$ matches all keys), while token 1 attends mostly to token 1 and token 3.

(c) Bayesian attention vs. deterministic attention.

The problem with deterministic attention. Standard softmax attention produces a single, fixed attention weight vector for each query. When multiple keys have similar relevance scores, softmax still commits to a (nearly) sharp distribution. This can be problematic:

- The model appears confident about which tokens are relevant, even when it is genuinely ambiguous.
- Small perturbations in keys can cause large changes in attention weights (sensitivity).

Bayesian attention. Instead of outputting a single attention weight vector, Bayesian attention maintains a *distribution* over attention weights. This distribution has:

- **Low entropy** when one key clearly dominates: the model is confident about where to attend.
- **High entropy** when multiple keys are similarly relevant: the model admits it is uncertain about the correct attention pattern.

What does this capture? Epistemic uncertainty in the attention pattern. Deterministic attention gives a point estimate of "how much to attend to each token." Bayesian attention gives a distribution, reflecting that with limited or ambiguous data, the correct attention pattern is itself uncertain.

Why it matters. In safety-critical applications (autonomous driving, medical diagnosis), knowing *where the model is uncertain about what to attend to* is as important as knowing the prediction. If the model is unsure whether a radar blip or a camera blob is more relevant, Bayesian attention flags this uncertainty.

11 Topic 11: Decision Making & Reinforcement Learning

R11: From Bayesian Decisions to Deep RL

(a) Define VPI. Why is VPI ≥ 0?
(b) Write the Bellman optimality equation for $Q^*(s, a)$.
(c) Q-learning: on-policy or off-policy? Explain maximization bias.

Solution

(a) Value of Perfect Information (VPI).

Revision: Bayesian decision theory. Given uncertain state of the world θ, action a, and utility $U(a, \theta)$, the **maximum expected utility** (MEU) is:

$$\text{MEU} = \max_a \ \mathbb{E}_\theta[U(a, \theta)].$$

VPI definition. The Value of Perfect Information is the expected gain from learning θ before deciding:

$$\text{VPI} = \mathbb{E}_\theta\big[\max_a U(a, \theta)\big] - \max_a \mathbb{E}_\theta[U(a, \theta)] = \text{MEU}_{\text{with info}} - \text{MEU}_{\text{without info}}.$$

With perfect information, you observe θ first, then pick the best action *for that* θ — you always make the right decision. Without information, you must choose a before seeing θ, which may be suboptimal.

Why $\text{VPI} \geq 0$**?** By Jensen's inequality for the convex function $\max_a$:

$$\mathbb{E}_\theta[\max_a U(a, \theta)] \geq \max_a \mathbb{E}_\theta[U(a, \theta)].$$

Intuitively: information can always be *ignored*. If the information tells you nothing useful, you just proceed as before. If it tells you something, you can adjust. So information never hurts in expectation.

Numerical example. Umbrella decision. $\theta \in \{\text{rain}, \text{sun}\}$, $p(\text{rain}) = 0.3$. Actions: bring umbrella ($U = 5$ rain, $U = 3$ sun) or don't ($U = 0$ rain, $U = 6$ sun). Without info: $\text{EU}(\text{umbrella}) = 0.3 \times 5 + 0.7 \times 3 = 3.6$; $\text{EU}(\text{no umbrella}) = 0.3 \times 0 + 0.7 \times 6 = 4.2$. $\text{MEU}_{\text{without}} = 4.2$. With info: if rain, choose umbrella ($U = 5$); if sun, choose no umbrella ($U = 6$). $\text{MEU}_{\text{with}} = 0.3 \times 5 + 0.7 \times 6 = 5.7$. $\text{VPI} = 5.7 - 4.2 = 1.5 > 0$.

(b) Bellman optimality equation.

The **optimal action-value function** $Q^*(s, a)$ gives the maximum expected cumulative discounted reward achievable from state s after taking action a:

$$\boxed{Q^*(s, a) = R(s, a) + \gamma \sum_{s'} p(s' \mid s, a) \max_{a'} Q^*(s', a').}$$

Interpretation. This is a recursive decomposition:

- $R(s, a)$: immediate reward from taking action a in state s.
- $\gamma \in [0, 1)$: discount factor (how much we value future vs. present rewards).
- $\sum_{s'} p(s' \mid s, a)[\cdot]$: expectation over stochastic transitions to next state s'.
- $\max_{a'} Q^*(s', a')$: after transitioning, we behave optimally from s' onward (the "optimal" in Q^*).

Why is this fundamental? The Bellman equation turns an infinite-horizon optimisation problem into a fixed-point equation. Methods like value iteration, Q-learning, and DQN all work by iteratively applying this equation.

Example. A two-state MDP: $s_1 \xrightarrow{a} s_2$ with $R = 1$, then $s_2 \xrightarrow{a} s_1$ with $R = 0$, $\gamma = 0.9$. At convergence: $Q^*(s_1, a) = 1 + 0.9 \times Q^*(s_2, a)$ and $Q^*(s_2, a) = 0 + 0.9 \times Q^*(s_1, a)$. Solving: $Q^*(s_1, a) = 1/(1 - 0.81) \approx 5.26$, $Q^*(s_2, a) = 0.9/(1 - 0.81) \approx 4.74$.

(c) Q-learning: off-policy and maximisation bias.

On-policy vs. off-policy. The Q-learning update is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big].$$

The target uses $\max_{a'} Q(s',a')$, which assumes we will act optimally in the future — regardless of what exploratory action the behaviour policy actually takes. This makes Q-learning **off-policy**: the learned value function corresponds to the optimal policy, not the policy used to collect data. In contrast, SARSA (an on-policy method) uses $Q(s',a')$ where a' is the action *actually taken*.

Maximisation bias. The $\max_{a'} Q(s',a')$ operator introduces a systematic upward bias. To see why, consider noisy Q-values with zero true value: $Q(s',a') = 0 + \epsilon_{a'}$ where $\epsilon_{a'} \sim \mathcal{N}(0,\sigma^2)$. Then:

$$\mathbb{E}[\max_{a'} Q(s',a')] = \mathbb{E}[\max_{a'} \epsilon_{a'}] > 0 = \max_{a'} \mathbb{E}[Q(s',a')].$$

This is Jensen's inequality applied to the convex function max. The more actions available and the noisier the estimates, the worse the overestimation. This can cause the agent to systematically prefer suboptimal actions that happen to have overestimated Q-values.

Fix: Double Q-learning. Maintain two independent Q-tables, Q_A and Q_B. Use Q_A to *select* the best action $a^* = \arg\max_{a'} Q_A(s',a')$, but Q_B to *evaluate* it: target $= r + \gamma Q_B(s',a^*)$. Since the selection and evaluation use independent estimates, the bias is eliminated. In DQN, Double DQN uses the online network to select and the target network to evaluate.

Part II
Cross-Topic Bridging Problems

12 The ELBO in Four Guises

R12 [Cross-Topic]: (Topics 5, 6, 9) The ELBO Everywhere

Identify for each context: the latent variables, q, whether q is learned, and the optimisation method.
(a) Mean-field VI (b) VAE (c) Bayes by Backprop (d) DDPM

Solution

Revision: why does the ELBO appear everywhere?
The ELBO is the universal tool for approximate Bayesian inference. Whenever we have latent variables $\mathbf{z}$ and want to maximise $\log p(\mathbf{x})$ (or equivalently, approximate the posterior $p(\mathbf{z} \mid \mathbf{x})$), we introduce $q(\mathbf{z})$ and optimise:

$$\mathcal{L}(q) = \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})] - D_{\mathrm{KL}}(q(\mathbf{z}) \| p(\mathbf{z})) \leq \log p(\mathbf{x}).$$

What changes across methods is: what are the latent variables, what family does q belong to, is q learned or fixed, and how do we optimise?

	Mean-field	**VAE**	**BbB**	**DDPM**
Latents	params $\mathbf{z}$	per-input $\mathbf{z}$	weights $\mathbf{w}$	$\mathbf{x}_{1:T}$
q	$\prod q_i(z_i)$	NN encoder	$\prod \mathcal{N}(\mu_j, \sigma_j^2)$	fixed forward
q learned?	Yes (CAVI)	Yes (SGD)	Yes (SGD)	**No**
Trick	coordinate ascent	reparam.	reparam.	noise pred.

(a) Mean-field VI.
Latent variables. The model parameters $\mathbf{z} = (z_1, \ldots, z_d)$ themselves. We want $p(\mathbf{z} \mid \mathbf{x})$ but it is intractable.
Variational family. Fully factored: $q(\mathbf{z}) = \prod_{i=1}^{d} q_i(z_i)$. Each factor is optimised independently, ignoring correlations between latent variables.
q **learned?** Yes, via **Coordinate Ascent Variational Inference (CAVI)**: iteratively update each q_i while holding the others fixed. The optimal $q_j^*(z_j) \propto \exp(\mathbb{E}_{q_{-j}}[\log p(\mathbf{z}, \mathbf{x})])$.
Key limitation. The factored assumption cannot capture posterior correlations. If z_1 and z_2 are strongly correlated a posteriori, mean-field q will underestimate the uncertainty along the correlated direction.

(b) VAE.
Latent variables. Per-input latent codes $\mathbf{z}$, one for each data point $\mathbf{x}$. These represent the "compressed representation" of the input.
Variational family. An *amortised* encoder network: $q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \mathrm{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x})))$. Instead of optimising a separate q for each data point (as in mean-field), a single neural network maps any input $\mathbf{x}$ to its approximate posterior parameters.
q **learned?** Yes, via SGD on the ELBO using the reparameterization trick.

Key insight. Amortisation trades per-input optimality (there is an "amortisation gap": the encoder cannot perfectly approximate every data point's posterior) for massive speedup (one forward pass at test time instead of iterative optimisation).

(c) Bayes by Backprop (BbB).
Latent variables. The neural network weights $\mathbf{w}$. We want $p(\mathbf{w} \mid \mathcal{D})$ but the weight-space posterior of a neural network is highly non-Gaussian and intractable.
Variational family. Mean-field Gaussian: $q_\phi(\mathbf{w}) = \prod_j \mathcal{N}(\mu_j, \sigma_j^2)$. Each weight has its own learned mean and variance.
q **learned?** Yes, via SGD using the reparameterization trick (same as VAE, but applied to weights instead of latent codes).
Key limitation. Doubles the number of parameters (μ_j and σ_j for each weight). The mean-field assumption ignores weight correlations, which can lead to poor uncertainty estimates.

(d) DDPM.
Latent variables. The entire noising trajectory $\mathbf{x}_{1:T}$ — a hierarchy of T latent layers, each progressively noisier.
Variational family. The forward diffusion process $q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ where $q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\mathbf{x}_{t-1}, (1-\alpha_t)\mathbf{I})$.
q **learned? No!** This is the unique feature of DDPM. The "encoder" (forward process) is entirely fixed by design — only the "decoder" (reverse process) $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is learned. Since q is fixed, there are no encoder parameters to optimise. The reparameterization trick is replaced by the noise-prediction objective $\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\|^2$.

Unifying insight. The reparameterization trick from Topic 5 is the computational engine behind VAE and BbB: it enables gradient-based optimisation of the ELBO through sampling operations. DDPM bypasses it by fixing q, which simplifies training but limits the expressiveness of the "encoder."

13 Langevin: From MCMC to Diffusion

R13 [Cross-Topic]: (Topics 3, 9) Score-Based Sampling Across the Course

(a) Write the Langevin update for $p(x) = \mathcal{N}(0, 1)$. Interpret the drift.
(b) Rewrite the DDPM reverse step using the score-noise equivalence. Compare with Langevin.
(c) Why does multi-scale noise make diffusion better than plain Langevin for complex distributions?

Solution

(a) Langevin dynamics for a standard normal.
Revision. The Langevin update (from Topic 3) for target distribution $\pi(x)$ is:

$$x_{k+1} = x_k + \frac{\eta}{2}\nabla_x \log \pi(x_k) + \sqrt{\eta}\, z_k, \quad z_k \sim \mathcal{N}(0, 1).$$

For $\pi(x) = \mathcal{N}(0,1)$, we have $\log \pi(x) = -x^2/2 + \text{const}$, so $\nabla_x \log \pi(x) = -x$. Substituting:

$$\boxed{x_{k+1} = x_k - \frac{\eta}{2} x_k + \sqrt{\eta}\, z_k = (1 - \eta/2)\, x_k + \sqrt{\eta}\, z_k.}$$

(Using the common convention $\eta/2$ for the drift, or equivalently writing $(1-\eta)x_k + \sqrt{2\eta}\, z_k$ with a slightly different parameterisation.)

Interpretation of the drift. The term $-\frac{\eta}{2} x_k$ pulls the sample toward 0 (the mean/mode of the target). If $x_k > 0$, the drift is negative (pull left); if $x_k < 0$, the drift is positive (pull right). The strength of the pull is proportional to the distance from the mean. This is exactly the behaviour of an Ornstein–Uhlenbeck (OU) process, whose stationary distribution is $\mathcal{N}(0,1)$.

The noise term $\sqrt{\eta}\, z_k$ provides **exploration**: without it, the chain would deterministically converge to 0 (the mode). The balance between drift (exploitation) and noise (exploration) ensures the chain samples from the full distribution, not just the mode.

Numerical example. Starting at $x_0 = 5$ with $\eta = 0.5$: drift $= -0.25 \times 5 = -1.25$; one step moves $x_1 \approx 5 - 1.25 + \sqrt{0.5} \times z \approx 3.75 + 0.71z$. The chain is pulled strongly toward 0. After many steps, it fluctuates around 0 with variance 1, sampling from the target.

(b) DDPM reverse step as a Langevin update.

Revision. The DDPM reverse step generates $\mathbf{x}_{t-1}$ from $\mathbf{x}_t$:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\, \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t\, \mathbf{z}.$$

Using the score–noise equivalence $\boldsymbol{\epsilon}_\theta \approx -\sqrt{1 - \bar{\alpha}_t}\, \nabla \log q(\mathbf{x}_t)$ from Topic 9:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} (\mathbf{x}_t + (1 - \alpha_t)\, \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)) + \sigma_t\, \mathbf{z}.$$

Comparison with Langevin:

	Langevin (Topic 3)	**DDPM reverse (Topic 9)**
Drift	$\frac{\eta}{2} \nabla \log \pi(\theta)$	$\frac{1-\alpha_t}{\sqrt{\alpha_t}} \nabla \log q(\mathbf{x}_t)$
Noise	$\sqrt{\eta}\, z$	$\sigma_t\, z$
Step size	fixed η	time-varying $(1 - \alpha_t)/\sqrt{\alpha_t}$
Target	fixed $\pi(\theta)$	time-varying $q(\mathbf{x}_t)$

Key differences. (1) DDPM uses a *scheduled* step size that varies with t, not a fixed η. (2) The target distribution changes at each step: at large t, $q(\mathbf{x}_t) \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$; as $t \to 0$, $q(\mathbf{x}_t)$ approaches the data distribution. Langevin targets a single fixed distribution throughout.

Interpretation. Both methods use "score + noise" to sample, but diffusion orchestrates this over a sequence of smoothly changing targets, while Langevin operates on one target. Diffusion is like a guided tour from simple to complex; Langevin is like exploring a fixed landscape.

(c) Why multi-scale noise helps.

The problem with plain Langevin on complex distributions. Consider a multimodal distribution with well-separated modes (e.g., two Gaussians 10 standard deviations apart). Between the modes, the density is near zero, so $\nabla \log p(\mathbf{x}) \approx \mathbf{0}$ (the gradient provides no useful direction). The Langevin chain gets "stuck" in one mode — transitioning between modes requires many steps of pure random walk through a region of negligible gradient signal.

How diffusion solves this. Diffusion operates at *multiple noise levels*:

- At large t (high noise): $q(\mathbf{x}_t) \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$ is nearly unimodal. The score function is well-defined everywhere and points clearly toward the origin. It is easy to sample.
- At intermediate t: modes begin to separate, but the noise smooths over the gaps. The score still provides useful gradient signal between modes.
- At small t (low noise): $q(\mathbf{x}_t) \approx p_{\text{data}}(\mathbf{x})$, the full multimodal structure emerges. But by now, the chain is already near a mode, so it only needs to refine locally.

This **noise curriculum** bridges simple and complex distributions. The chain transitions between modes at high noise levels (where it is easy) and refines within modes at low noise levels (where precision matters).

Analogy. This is closely related to **simulated annealing** in optimisation: start at high temperature (explore broadly), gradually cool (exploit locally). Diffusion does this for *sampling* rather than optimisation.

14 Precisions Add: The Unifying Principle

R14 [Cross-Topic]: (Topics 1, 2, 4, 8) Gaussian Fusion Everywhere

Show that the same principle governs: (a) Gaussian mean updating, (b) Bayesian linear regression, (c) Kalman filter update, (d) Two-sensor fusion.

Solution

Revision: the core principle.
Whenever we combine Gaussian information sources using Bayes' rule, the result follows one universal pattern:

$$\boxed{\text{posterior precision} = \text{prior precision} + \text{data precision}.}$$

Equivalently, the posterior mean is a *precision-weighted average* of the prior mean and the data. This is arguably the single most important result in the course, and it appears in at least four different contexts.

(a) Gaussian mean updating (Topic 1).
Suppose $x_1, \ldots, x_n \mid \mu \sim \mathcal{N}(\mu, \sigma^2)$ with prior $\mu \sim \mathcal{N}(\mu_0, \sigma_0^2)$. Define prior precision $\lambda_0 = 1/\sigma_0^2$ and data precision per observation $\lambda_{\text{obs}} = 1/\sigma^2$. After n observations:

$$\lambda_N = \lambda_0 + n\lambda_{\text{obs}} = \frac{1}{\sigma_0^2} + \frac{n}{\sigma^2}, \qquad \mu_N = \frac{\lambda_0\mu_0 + n\lambda_{\text{obs}}\bar{x}}{\lambda_N}.$$

Example. Prior: $\mu_0 = 0$, $\sigma_0^2 = 4$ ($\lambda_0 = 0.25$). Data: $n = 5$ observations with mean $\bar{x} = 3$, $\sigma^2 = 1$ ($\lambda_{\text{obs}} = 1$). Then $\lambda_N = 0.25 + 5 = 5.25$, $\mu_N = (0.25 \times 0 + 5 \times 3)/5.25 = 15/5.25 \approx 2.86$. The posterior is pulled strongly toward the data because $5\lambda_{\text{obs}} \gg \lambda_0$.

(b) Bayesian linear regression (Topic 2).
With prior $\mathbf{w} \sim \mathcal{N}(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$ and likelihood $\mathbf{y} \mid \mathbf{X}, \mathbf{w} \sim \mathcal{N}(\mathbf{X}\mathbf{w}, \sigma^2\mathbf{I})$:

$$\boldsymbol{\Sigma}_N^{-1} = \boldsymbol{\Sigma}_0^{-1} + \sigma^{-2}\mathbf{X}^\top\mathbf{X}.$$

This is the matrix generalisation of (a). Each data point $\mathbf{x}_i$ contributes $\sigma^{-2}\mathbf{x}_i\mathbf{x}_i^\top$ to the precision matrix. Directions in weight space that are well-observed (aligned with many $\mathbf{x}_i$) have high precision; poorly observed directions retain prior uncertainty.

(c) Kalman filter update (Topic 4).
Scalar case. Predict: $P_{t|t-1}$ (predicted uncertainty). Observation: $y_t = Hz_t + r_t$, $r_t \sim \mathcal{N}(0, R)$. The information form of the Kalman update is:

$$P_{t|t}^{-1} = P_{t|t-1}^{-1} + H^2/R.$$

This is exactly "precisions add": the predicted precision $P_{t|t-1}^{-1}$ plays the role of the prior, and the measurement precision H^2/R plays the role of the data.
Matrix case. $\mathbf{\Sigma}_{t|t}^{-1} = \mathbf{\Sigma}_{t|t-1}^{-1} + \mathbf{H}^\top\mathbf{R}^{-1}\mathbf{H}$.
Connection. Comparing with (b): $\mathbf{\Sigma}_{t|t-1}^{-1}$ corresponds to $\mathbf{\Sigma}_0^{-1}$ (prior), and $\mathbf{H}^\top\mathbf{R}^{-1}\mathbf{H}$ corresponds to $\sigma^{-2}\mathbf{X}^\top\mathbf{X}$ (data), but with $\mathbf{X} = \mathbf{H}$ (single observation) and $\sigma^2 = R$. The Kalman filter is just sequential Bayesian linear regression, one observation at a time.

(d) Two-sensor fusion (Topic 8).
Two sensors measure the same quantity: $y_1 \sim \mathcal{N}(z, \sigma_1^2)$, $y_2 \sim \mathcal{N}(z, \sigma_2^2)$. The fused precision is:

$$\lambda_f = \lambda_1 + \lambda_2 = \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}.$$

This is the special case of (a) with no prior (or a flat prior $\sigma_0^2 \to \infty$, $\lambda_0 = 0$) and two observations with different noise levels.

The unifying view.
In all four cases, more precise information (lower variance = higher precision) receives higher weight in the posterior mean. The posterior is always more precise than any single source. This principle extends beyond the course to any product of Gaussians:

$$\mathcal{N}(\mu_1, \sigma_1^2) \times \mathcal{N}(\mu_2, \sigma_2^2) \propto \mathcal{N}(\mu_f, \sigma_f^2), \qquad \lambda_f = \lambda_1 + \lambda_2, \qquad \mu_f = \frac{\lambda_1\mu_1 + \lambda_2\mu_2}{\lambda_f}.$$

15 Aleatoric vs. Epistemic: The Full Picture

R15 [Cross-Topic]: (All Topics) Uncertainty Decomposition Across Eight Methods

For each method state whether it captures aleatoric, epistemic, or both, and how.

Solution

Revision: the two fundamental types of uncertainty.
Aleatoric (irreducible) uncertainty arises from inherent randomness in the data-generating process: sensor noise, quantum effects, uncontrollable environmental variation. No amount of data can eliminate it. It is a property of the *world*.

Epistemic (reducible) uncertainty arises from our *ignorance*: insufficient data, model misspecification, unknown parameters. More data, better models, or additional information can reduce it. It is a property of our *knowledge*.
The distinction matters for decision-making: epistemic uncertainty suggests we should gather more data; aleatoric uncertainty means we should accept the noise and design robust systems.

Method	Type	How
Bayesian regression	Both	σ^2 (ale.) + $\mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*$ (epi.)
Kalman filter	Both	Q, R (ale.) + $P_{t\|t}$ shrinks with data (epi.)
Particle filter	Both	Same as KF but handles non-Gaussian (multimodal)
MC Dropout + hetero.	Both	Variance across passes (epi.) + mean of $\sigma^2(\mathbf{x})$ (ale.)
Conformal prediction	Neither	Marginal coverage guarantee; no decomposition
Diffusion (single)	∼Aleatoric	Sample diversity ≈ data ambiguity; ensemble for epi.
Deep ensemble	Both	Inter-model disagreement (epi.) + mean σ^2 heads (ale.)
UCB in RL	Epistemic	Bonus $\propto \sqrt{\ln t/N(a)}$ shrinks with observations

Row-by-row explanation.
Bayesian regression. The predictive variance $\sigma^2 + \mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*$ is an additive decomposition: σ^2 is observation noise (aleatoric), and $\mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*$ is weight uncertainty (epistemic). As $N \to \infty$, $\boldsymbol{\Sigma}_N \to \mathbf{0}$, and only σ^2 remains.
Kalman filter. The process noise Q and measurement noise R are aleatoric — they persist regardless of how long we observe. The estimation error covariance $P_{t|t}$ reflects how well we have estimated the state; it shrinks as we accumulate measurements (epistemic). In steady state, $P_{t|t}$ converges to a fixed value that depends on the Q/R ratio.
Particle filter. Same conceptual decomposition as the Kalman filter, but the particle representation can capture non-Gaussian (multimodal) posteriors. Epistemic uncertainty is reflected in the spread and diversity of particles.
MC Dropout + heteroscedastic head. Run T forward passes with dropout active. Each pass gives a mean $\mu_t(\mathbf{x})$ and variance $\sigma_t^2(\mathbf{x})$. Epistemic $\approx \frac{1}{T}\sum_t (\mu_t - \bar{\mu})^2$ (disagreement between passes). Aleatoric $\approx \frac{1}{T}\sum_t \sigma_t^2(\mathbf{x})$ (average predicted noise).
Conformal prediction. Conformal prediction provides marginal coverage guarantees $\Pr(y \in C(x)) \geq 1 - \alpha$ but does *not* decompose uncertainty into aleatoric and epistemic. It is a distribution-free wrapper that guarantees validity, not a model of uncertainty sources.
Diffusion (single model). A single diffusion model generates diverse samples; this diversity roughly reflects the ambiguity in the data (aleatoric). However, a single model cannot distinguish "I generate diverse samples because the data is genuinely ambiguous" from "I generate diverse samples because I am uncertain about my parameters." For epistemic uncertainty, one can use an ensemble of diffusion models and measure inter-model disagreement.
Deep ensemble. Train M independently initialised networks. Epistemic uncertainty ≈ variance of predictions across models (disagreement). Aleatoric uncertainty ≈ mean of individual models' predicted variances (if using heteroscedastic heads). Ensembles are simple, effective, and widely used in practice.
UCB in RL. Upper Confidence Bound selects actions via $a_t = \arg\max_a [Q(a) + c\sqrt{\ln t/N(a)}]$. The bonus $c\sqrt{\ln t/N(a)}$ is purely epistemic: it is large for rarely-tried actions ($N(a)$ small) and shrinks as the action is explored. UCB does not model aleatoric uncertainty (reward noise).

16 Calibration Propagates Through Pipelines

R16 [Cross-Topic]: (Topics 4, 7, 8, 11) Miscalibration in a Safety-Critical System

A perception module feeds a Kalman tracker, then sensor fusion, then a brake/don't-brake decision. The detector is overconfident: $\hat{p} = 0.95$, true $p^* = 0.70$.
(a) Effect on Kalman update? (b) Effect on fusion? (c) Can it flip a decision? (d) How does conformal prediction help?

Solution

Revision: why calibration matters in pipelines.
In a safety-critical pipeline (autonomous driving, medical diagnosis), multiple modules consume each other's uncertainty estimates. If an upstream module's uncertainties are wrong (miscalibrated), the errors propagate and amplify downstream. This problem connects calibration (Topic 7), Kalman filtering (Topic 4), sensor fusion (Topic 8), and decision theory (Topic 11).

(a) Effect on the Kalman update.
The Kalman filter uses the measurement noise covariance R to decide how much to trust each observation. The Kalman gain is $K = P_{t|t-1}/(P_{t|t-1} + R)$ (scalar case).
If the detector is **overconfident**, it reports uncertainty that is too small — equivalently, R is set too low.
Consequences:

- R too small $\Rightarrow$ $K = P/(P+R)$ too large (approaches 1).
- The filter assigns excessive weight to the measurement, treating noisy data as highly reliable.
- The updated state $\hat{z}_{t|t} = \hat{z}_{t|t-1} + K(y - \hat{z}_{t|t-1})$ is pulled too strongly toward each measurement, making the filter **over-reactive** to noise.
- The updated covariance $P_{t|t} = (1-K)P_{t|t-1}$ becomes artificially small: the filter thinks it knows the state better than it actually does.

Example. True $R = 4$, but detector reports $R = 0.25$ (overconfident by 16×). With $P_{t|t-1} = 3$: true $K = 3/7 \approx 0.43$; overconfident $K = 3/3.25 \approx 0.92$. A noisy measurement pulls the state estimate almost entirely to the observation.

(b) Effect on sensor fusion.
In precision-weighted fusion (Topic 8), each sensor's weight is $w_i \propto \lambda_i = 1/\sigma_i^2$. An overconfident sensor reports σ_i^2 too small, so λ_i is inflated.
Consequence: the overconfident sensor receives disproportionately high weight in the fused estimate. If this sensor's measurement happens to be wrong (due to noise, occlusion, or malfunction), the fused estimate is **biased toward the wrong value**.
Example. Sensor A (overconfident): $y_A = 3$, reported $\sigma_A = 0.5$ (true $\sigma_A = 2$). Sensor B (well-calibrated): $y_B = 7$, $\sigma_B = 1$. Reported fusion: $w_A = 4/(4+1) = 0.8$, $\hat{z} = 0.8 \times 3 + 0.2 \times 7 = 3.8$. True fusion (with correct $\sigma_A = 2$): $w_A = 0.25/(0.25+1) = 0.2$, $\hat{z} = 0.2 \times 3 + 0.8 \times 7 = 6.2$. The overconfident sensor dragged the fused estimate from 6.2 to 3.8 — a massive error.

(c) Can miscalibration flip a decision?
Yes. Consider the brake/don't-brake decision using expected utility. Let p be the probability of a pedestrian being present.
Costs: braking (false alarm) costs 80, hitting a pedestrian costs 100.

At the true probability $p^* = 0.30$ (since $1 - p^* = 0.70$): Expected cost of braking: 80. Expected cost of not braking: $0.30 \times 100 + 0.70 \times 0 = 30$. Since $30 < 80$: optimal action is **don't brake**.
At the overconfident reported probability $\hat{p} = 0.95$**:** Expected cost of braking: 80. Expected cost of not braking: $0.95 \times 100 + 0.05 \times 0 = 95$. Since $95 > 80$: optimal action is **brake**.
The miscalibrated confidence has **flipped the decision** from "don't brake" to "brake." In the reverse scenario (under-confident detector in a real emergency), the decision could flip toward "don't brake" when braking was necessary — with potentially fatal consequences.
Key insight. Decision theory amplifies miscalibration: even moderate probability errors can cross decision thresholds and cause qualitatively wrong actions.

(d) How conformal prediction helps.
Conformal prediction (Topic 7) provides a **distribution-free, model-agnostic** safety net:

- Instead of trusting the detector's probabilities, conformal prediction produces a **prediction set** $C(x)$ with guaranteed coverage: $\Pr(y_{\text{true}} \in C(x)) \geq 1 - \alpha$.
- If $C(x) = \{\text{pedestrian}\}$: the detector is confident and the conformal set confirms it — proceed with the calibrated decision.
- If $C(x) = \{\text{pedestrian, no pedestrian}\}$: the conformal set says the situation is genuinely ambiguous. The system should default to the **safe action** (brake), regardless of the detector's potentially miscalibrated probability.
- If $C(x) = \{\text{no pedestrian}\}$: the detector and conformal prediction agree there is no pedestrian.

Why this is powerful. The conformal guarantee holds regardless of the model's calibration. Even if the detector is badly miscalibrated, the conformal set still has valid coverage. It acts as a "sanity check" on the detector's output.

17 Five Ways to Approximate a Posterior

R17 [Cross-Topic]: (All Topics) A Taxonomy of Posterior Computation

For each strategy, name the course method(s) and state the main trade-off.
(a) Exact conjugate (b) Deterministic approx. (c) MCMC (d) Amortised inference (e) Iterative denoising

Solution

Revision: the central problem of Bayesian inference.
Given data $\mathcal{D}$ and a model with parameters or latents $\mathbf{z}$, we want the posterior $p(\mathbf{z} \mid \mathcal{D}) = p(\mathcal{D} \mid \mathbf{z})p(\mathbf{z})/p(\mathcal{D})$. The denominator $p(\mathcal{D}) = \int p(\mathcal{D} \mid \mathbf{z})p(\mathbf{z})\, d\mathbf{z}$ (the evidence) is almost always intractable. The entire course can be viewed as five progressively more general strategies for dealing with this intractability.

(a) Exact conjugate.
Methods: Beta–Binomial (Topic 1), Gaussian–Gaussian (Topic 2, Bayesian linear regression with known σ^2).

How it works. When the prior belongs to the conjugate family for the likelihood, the posterior is available in closed form. No approximation is needed.
Trade-off. Fast and exact, but only works for a very restricted set of model families (exponential family with conjugate priors). Cannot handle neural networks, nonlinear models, or complex likelihoods.
Example. Beta$(2, 2)$ prior + 7 heads in 10 flips $\rightarrow$ Beta$(9, 5)$ posterior. Computed instantly, no iteration needed.

(b) Deterministic approximation.
Methods: Laplace approximation (Topic 3, logistic regression), mean-field VI (Topic 5), Bayes by Backprop (Topic 6).
How it works. Approximate the posterior with a simpler distribution (Gaussian at the mode for Laplace; factored distribution for mean-field; diagonal Gaussian per weight for BbB). Optimise the approximation by maximising the ELBO or minimising KL divergence.
Trade-off. Scalable to large models (SGD-based optimisation), but the approximation is inherently limited: Laplace captures only the local curvature at the mode; mean-field ignores correlations; reverse KL is mode-seeking and may miss modes.
Example. Laplace approximation for logistic regression: find $\hat{\mathbf{w}}_{\text{MAP}}$, compute the Hessian $\mathbf{H}$, approximate posterior as $\mathcal{N}(\hat{\mathbf{w}}, \mathbf{H}^{-1})$.

(c) MCMC.
Methods: Metropolis–Hastings (random walk), MALA (Topic 3), particle filter (Topic 4).
How it works. Generate a Markov chain whose stationary distribution is the exact posterior. Given enough samples, any posterior expectation can be estimated to arbitrary accuracy.
Trade-off. Asymptotically exact (no approximation error in the limit), but can be **very slow**: long burn-in, poor mixing in high dimensions. Particle filters are sequential MCMC for state space models but suffer from weight degeneracy in high dimensions.
Example. MALA for a 10-dimensional logistic regression posterior: 10,000 samples, 1,000 discarded for burn-in, effective sample size $\sim$2,000.

(d) Amortised inference.
Methods: VAE encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ (Topic 6).
How it works. Train a neural network (the encoder) to map any input $\mathbf{x}$ directly to the approximate posterior parameters. At test time, a single forward pass gives the approximate posterior — no iterative optimisation or sampling needed.
Trade-off. Very fast at test time (one forward pass), but the encoder cannot perfectly approximate every data point's posterior. This "amortisation gap" means the approximation is worse than per-input optimisation. The quality is also limited by the variational family (typically diagonal Gaussian).
Example. A VAE encoder maps a 28×28 image to $(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ in a 20-dimensional latent space in one forward pass ($\sim$1 ms).

(e) Iterative denoising.
Methods: DDPM (Topic 9), Diffusion Posterior Sampling (DPS).
How it works. Start from pure noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and iteratively denoise using learned score functions, gradually transforming noise into a sample from the target distribution.
Trade-off. Handles complex, multimodal distributions that simpler methods (Laplace, mean-field) cannot represent. But very **expensive**: requires $T\times$ (number of samples) neural network evaluations. Also approximate: the learned denoiser is not perfect, and the discretisation introduces error.

Example. Generating a 256×256 image: $T = 1000$ denoising steps, each requiring one forward pass through a U-Net. For $S = 50$ samples: 50,000 neural network evaluations.

The course arc.
The progression from (a) to (e) tells the story of the course: we start with exact, restrictive solutions and progressively relax assumptions, trading exactness for generality and computational cost.

Strategy	**Exactness**	**Generality**	**Cost**
Exact conjugate	Exact	Very limited	Negligible
Deterministic	Approximate	Moderate	Low–moderate
MCMC	Asymp. exact	General	High
Amortised	Approximate	General	Low at test
Denoising	Approximate	Very general	Very high

Acknowledgements / Document Preparation

The problems and solutions in this document were generated with assistance from large language models (ChatGPT by OpenAI, CoPilot by Microsoft, and Claude by Anthropic), based on the course slide and notebook material. The instructor reviewed, modified, and verified all questions, solutions, and final formatting.